High Performance Computing Co-Design Space I : Application Complexity Design space, Architectural Design Space and Parallel Programming Design Space

Anirudh Seshadri
Department of Information Technology
SSN College of Engineering
Chennai, India
anirudhseshadri95@gmail.com

Dhanasekar Sundararaman
Department of Information Technology
SSN College of Engineering
Chennai, India
dhanasekar312213@gmail.com

Sudharshan Srinivasan
Department of Information Technology
SRM College of Engineering
Chennai, India
sudharshan.srini@gmail.com

***Abstract*— Several research project reports and papers have been published advocating co-design methodology for developing low power high-performance computing systems for grand challenge applications. These research works no doubt meticulously focused on every aspect of co-design principles associated with application software, many-core architecture, stressing more on memory design, parallel programming models, languages, and low power technology down to 10nm.In fact most of the research reports elaborate more on memory design and technology(to reduce the data movement) which is the paramount driving force. But none of these research reports and papers have come out with an integrated solution platform characterizing application complexity design space, architecture design space, parallel language design space, simulator design space, optimizer design space and co-design automation of heterogeneous many core architecture for co-applications. An execution model, called All Embracing Execution for many core architecture for co-applications, without space time sharing unlike the multi-programming (on which the traditional co-design methodology is based), is proposed and is shown to achieve excellent resource utilization hence performance given a power cap over the multi-programming. Also dynamic memory bank allocation and cache replacement policies under All Embracing Execution model, need to be aware of runtime dynamics of co-applications. This is shown to further enhance the resource utilization to a greater extent. The published papers and reports don't deal with simulators and optimizers in the mind set of co-design of heterogeneous many core architecture for co-applications under All Embracing Execution. This paper characterizes application complexity design space, architecture design space and parallel language design space and their integrated interaction to provide a solution for co-design of heterogeneous many core architecture for co-applications, through a binding process. There are two other companion papers one on simulator-optimizer design spaces and the other on co-design automation of many core architecture for co-applications. The former is on All Embracing Execution based simulator-optimizer and the later is on the co-design automation of heterogeneous many core architectures.**

***Keywords*—**

I. INTRODUCTION

One of the earliest Multi-core architectures to have been built was the Raw Architecture Workstation with rather simpler features[1]. It has the ability to execute instructions out of order and multiple threads yield a better performance; the Intel Family Of Cores include an Out Of Order Execution Engine as a major component of the core; a typical example being the Intel Core i7 [2]. The Zen core from AMD is based on a superior FINFET technology and is expected to give a very high performance [3]. Yet another interesting, along the similar lines is the IBM Power8 which includes error handling capabilities, most used as server processors[4].Another domain which requires high performance computing is the graphics rendering and Nvidia has taken a great lead in this aspect. The Nvidia CUDA cores though being very simple, are grouped in large numbers as

Streaming Multi Processors making up the Graphic Processors. This makes them to be extensively used for gaming and editing purposes. The complexity of the graphic processors in terms of the number of cores has reached its current maximum of 3584 cores in the Nvidia GP100 processor[5].Targeting very high-performance computing, Intel has developed the Xeon Phi made up of cores that are composed of Vector Processor Units and Extended Math Units unlike most of the other vendors. The primary motivation for designing such a core architecture is to achieve a very high computation performance [6].On a different scale targeting Exa-Scale Computing, Intel has launched a major research initiative called the Traleika Glacier Project which led to the construction of a new processor architecture called the X-Stack. It is a block based architecture and consists of several Execution Engines (XEs), each accompanied with a local cache and L1 scratch pad. The XE consists of ALUs of variable word lengths ranging from 16-bit to 64-bit. The processor-chip is built with 32 such blocks and also contains Power monitors, Circuit Monitors and Powerful Networking System. This processor chip due to the sheer hardware power is expected to deliver higher performances with better Programming Models[7-11].The development of powerful cores from the above stated vendors are mostly driven by technology for achieving high performance and reduced energy consumption.

There exists no generic methodology of designing these cores. A generic and practical approach of developing cores meant for high performance computing systems should be based on the process of binding the Application, Architecture and the Programming Languages. At best, some of these vendors have developed suitable programming models for an array of processors for efficient mapping of the program sections onto the processors built around these cores[7][8]. The co-design of heterogeneous many core architecture involves the application complexity , architecture characteristics and the parallel programming languages to envision low power HPC system capable of executing thousands of petaflop operations per second. In this perspective a lot of research papers have been published concerning application complexity, heterogeneous many core architecture and parallel programming paradigm. More papers get published with regard to many core architecture (henceforth many core architecture means heterogeneous many core architecture) unlike in the domain of application complexity analysis and parallel programming paradigm, from perspective of co-design principles. The reason may be that there are more conceptual parameters, design and technological aspects associated with the many core architecture. All these related research work are briefed in this introduction section. Many papers suggest co-design for application specific architecture development and similarly there are papers pertaining to domain specific languages.

The traditional co-design [25,26,27,21, 22] approach lacks several important design aspects which are conceptualized in this paper.

A. The Synergy of design spaces:

To bring out the inherent synergy among the application complexity , many core architecture characteristics and the parallel programming paradigm to accomplish the co-design process, it is pivotal to establish pertinent links across the application complexity design space, many core architecture design space and parallel programming language design space. Currently many core architecture design space, its relevant attributes and the design space exploration are very well established. On the other hand application complexity design space and parallel programming language design space are not comprehensively modeled, though their basic attributes are known. From the perspective of modeling these two design spaces, some more attributes need to be included and these are explained under respective sections in this paper. The interaction among the three design spaces during the co-design process is explicitly brought out and discussed in this and the two more companion papers.

B. Power-performance efficient many core architecture: contributing factors

To meet the computational demands exceeding thousands of petaflops, for solving grand challenge applications[30-32,16,36], there has been a sustained and enormous efforts by the HPC community around the globe, to research into the various aspects of HPC system like the homogeneous/ heterogeneous many core architecture, types of core, architecture design space exploration, instruction set for many core heterogeneous architecture, efficient memory bank allocation, cache hierarchy, heterogeneous network on chip, task scheduling, thread/task/application migration across the cores, programming models, and application complexity analysis to design and develop energy efficient high performance architecture. Several research papers, reports and white papers are getting published [16-24] to arrive at viable solutions to build low power HPC systems. Co-design [25-39,21,22] has been the focus of research to accomplish this complex goal.

1. Co-design:

Co-design is a methodology in which a logical binding of application, architecture and the programming language characteristics are comprehended. A major research activity in this direction has been initiated [22] since a decade. Apart from the number of published papers, several co-design research initiatives are carried out, the notable are the Traleika Glacier X-Stack [66,88] from the Intel, the “RAMP” project[36] from UC Berkely in association with the Lawrence Berkely National Lab and from Domain Specific Computing Center, UCLA[35,37].The Tensilica design tools, though meant for developing low energy processor based on Xtensa architecture[34] primarily for real time embedded systems, these tools have been successfully applied to develop the Green Flash an extreme-scale computing system for atmospheric

modeling [16],following the concepts of co-design principles.

2. Extreme parallelism/Accelerators/Legacy code:

In the near future HPC system will contain hundreds of thousands cores. Instead of instruction level parallelism and OOO execution[19],extreme parallelism architectures (heterogeneous many core) should be based on the concept of explicit form of parallelism akin to SIMD class or even MIMD class to surmount the power-performance bottle neck[19].The power-performance criteria of the present HPC system should get shifted to 50Gflops/W from the current value of 2Gflops/W. Further to this, majority of applications related, large scale software packages are unsuitable[18] to take full advantage of extreme parallelism of the future heterogeneous many core architectures which may include special purpose functional units as an integral part of the cores besides on chip stand alone accelerators[19-21]. Accelerators are used to achieve energy efficiency, in spite of the impediments associated with them which are elaborated later. To make the legacy code portable on these system("cores+ specialized cores + accelerators/coprocessors")is a very complex process and its pros and cons are discussed in number of papers [18,23,26] either explicitly or implicitly. "Increasing gaps between the capabilities of today's programming environments, the requirements of emerging applications, and the challenges of future parallel architectures" will lead to the redesigning of contemporary software.

3. Scheduling/migration:

Several billions of devices[53] get integrated in a many core based processors . FinFET based technology has advanced to an extent to accommodate millions of cores to build an energy efficient supercomputing clusters. To exploit the extreme parallelism architectures, capable of executing billions of operations per second [23] the grand challenge applications need to be effectively parallelized. On the other hand during execution the different phases of an application may involve varying degree of parallelism to the extent of just serial processing alone. This leads to under utilization of resources and it is dealt with by pertinent scheduling processes including task/thread/application migration across the cores. There are several research papers on different class of scheduling and tasks/application migration across the heterogeneous many cores [18,25]

4. Memory/on chip data movement:

On chip data movement is a source of high energy inefficiency and on several occasions this energy is much more than that consumed by computations performed[19].Data movement [19,23,] is very much associated with memory system design, its bandwidth and memory access pattern [19,29].

5. Core architecture:

Heterogeneous many-core architectures are designed either for In-Order-Execution, Out-Of Order Execution or a combination of both. Implication of these on power-performance balancing are presented in.In particular the "big.LITTLE" core technology[94] promises highly energy efficient execution. Besides the core architecture framework, the ISA is a dominant factor in power-performance balancing. Several papers dealing with single ISA, heterogeneous ISA ,extendable ISA and customizable ISA have been published highlighting the merits and demerits in the context of heterogeneous many core architecture design.

II. INEFFICIENCY OF MULTIPROGRAMMING ACROSS MANY CORE ARCHITECTURE

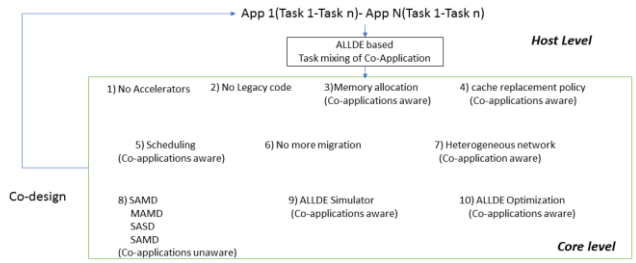
Multiprogramming operates on the principle of space-time sharing of the cores and their resources. In the near future either homogeneous or heterogeneous core count will exceed 100 millions. With very large number of cores, poor resource utilization may lead to highly inefficient power-performance balancing. In multiprogramming of tasks of different applications, across hundreds of thousands cores, large number of cores may remain ideal(of course the cores may be switched off, but still it is a big overhead). As pointed earlier there are diversified class of scheduling algorithms and thread/task/application migration techniques to maximize the resource utilization [18,25]. But the fundamental deficiency of multi-programming cannot be obliterated

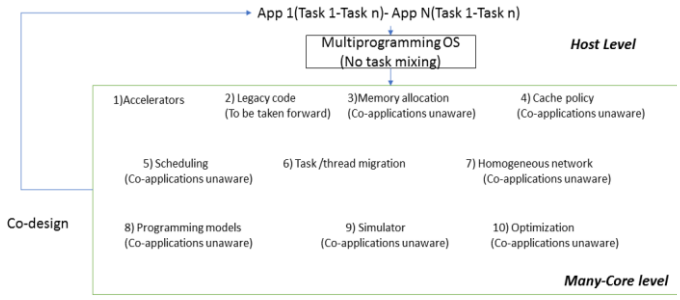
1. Overcoming the multiprogramming inefficiency: ALLDE

The concept of simultaneous execution of co-applications without space-time sharing has been proposed in [98] at node level to increase the resource utilization**.** Instructions of the same type, but from different applications can get executed in the pipeline stages of the same functional unit simultaneously at intra-core level. This is a first of its kind concept proposed at the intra-core level in .This concept is adopted in the proposed co-design of many core architecture for co-applications for the execution of these applications without space-time sharing which is shown to increase the resource utilization to a very large extent. This execution is termed as **All E**mbracing **E**xecution, ALLDE (ALL **E E** –ALL Double E—ALLDE). ALLDE model is totally different from the concept of co-scheduling presented in.These functional units are algorithm level functional units and scalars supported by algorithm level instruction set architecture; this ISA is designed to perform the ALLDE. More on this is deliberated in the section V.

III. PROPOSED METHODOLOGY FOR CO-DESIGN OF MANY CORE ARCHITECTURE FOR CO-APPLICATIONS

The proposed co-design of many core architecture for co-applications is based on the concept of **ALLDE** model unlike the traditional co-design methodology and is driven by several innovative concepts. Refer to Fig.1a and Fig.1b.





The immediate impact of ALLDE is that a tremendous improvement in resource utilization over multi-programming.

- Co-applications aware memory bank allocation during execution is important; run time dynamics of the co-applications are captured for power efficient allocation of memory banks. Even in the long established multiprogramming concept there is no co-application/co-tasks aware memory bank allocation.
- Co-applications aware cache replacement policies are crucial; run time dynamics of the co-applications are captured to tune the cache replacement policies which increase the cache hit ratio over the traditional.
- The intra core data transfer across application specific functional units/algorithm level functional units and scalars are through the On Core Network (OCN).This further increases the cache hit ratio.
- Inter core communication statistics obtained during simulation is leveraged for inherently forming the inter-core heterogeneous network architecture.
- The ALLDE needs efficient tasks mixing of co-applications at the host. Because of ALLDE, tasks of co-applications are run with in each core. This alleviates the necessity for complex scheduling algorithms ,core migration and core fusion processes.

1. Accelerators:

"The concept of accelerators as a separate co-processor will disappear in the near future; such a deep change in the processor architecture will require much more hardware-aware code in order to exploit all the available computational power"[20].The papers introduce the concept of Algorithm Level Functional Units which can be the constituents of intra core architecture and the associated instruction set, Algorithm Level Instruction Set Architecture, for designing power-performance efficient high performance computing system. In the proposed co-design of many core architecture for co-applications, customizable intra core components are, a combination of algorithm level functional units, the scalars and on core network (besides the cache levels), which are library like building blocks housed in the many core architecture design space. With processors built around these type of heterogeneous cores (bound to the co-

application characteristics –computational structures), in effect the accelerators are broken down and distributed across these heterogeneous cores in the form of algorithm level functional units and scalars in conformity with the characteristics of co-applications. This ushers in a deep change in the core/processor architecture[20] totally away from the traditional one to achieve the strongest binding across the co-applications and the many core architecture to realize the best performance given the power cap. More on this is presented in the subsequent sections. Farther, to accomplish the best resource utilization of these heterogeneous cores under ALLDE, much more "hardware-aware" efficient parallel programming model and parallel constructs need to be developed and this discussed subsequently.

2. Top-Down Parallel Programming Model: From the Host to Intra Core execution.

A new class of parallel programming model is defined to support the ALLDE at the host level, based on the mix of co-applications.

a) Single Application Multiple Data ---- SAMD. Climate modeling [16] multiple regions: In general the model (Single application) remains the same except the parameters value for different regions (multiple data).
b) Multiple Application Single Data ---- MASD. Large scale image (eg. of a far off planet) processing : The image(single data) of the heavenly object can under go simultaneous processing for enhancement, edge detection and segmentation.
c) Multiple Application Multiple Data----MAMD. Human brain region specific modeling: Neuronal Energetics simulation[106], Cerebral Blood Flow dynamics and Time-Frequency analysis of the neuronal spike activities---towards brain disease modeling and effective drug design. At the intra core level to support ALLDE is---MIMD. Refer Fig.2.

3. Legacy code:

In the above sub sections the complexity of porting the legacy code and its implication (power-performance inefficiency) are briefed citing several published papers. "Increasing gaps between the capabilities of today's programming environments, the requirements of emerging applications, and the challenges of future parallel architectures" will lead to the redesigning of contemporary software. On the other hand re-designing and rewriting this legacy code may be a time consuming process; but it seems worth the effort as the ultimate focus is to maximize the resource utilization of millions of cores to boost the performance under a given power cap. However the proposed parallel programming constructs will greatly simplify the legacy code rewriting as they are at algorithm level supported by scalar and other class of constructs based on C++ paradigm.

4. Co-applications aware memory system Design:

Both memory bank allocation and the cache system design should be well related (awareness) to the run time dynamics of co-applications. Currently cache replacement policies are decided unaware of co-applications execution status. Under the AALDE execution these policies are determined based on the run time statistics of the co-application and this is shown to boost the cache hit very much.

5. Inter-Core Communication Complexity:

Run time inter core communication complexity is modeled from the simulation results to design the heterogeneous inter-core network structures. Through these factors the many core architecture features get tightly bound to the dynamic characteristics of co-applications.

6. Parallel Programming Customizable Constructs:

Parallel programming constructs are based on the concept of "hardware-aware" code and these constructs can be customized completely to confirm with the heterogeneous intra core components. The programming model MPMD is a sub-set under the above classification; however it is obvious that the model SPMD will not fall under these classifications. The SAMD under co-applications is bound to have several programs running simultaneously.

IV. CO-DESIGN SPACES

Applications are characterized by certain fundamental factors like computational complexity and communication complexity besides control flow complexity and locality of reference. These are major contributing parameters towards efficient power-performance balancing under a power cap. Generally applications are constituted by different class of algorithms like numeric ,semi-numeric , non-numeric and a complex combination of all these along with general purpose operations. Thus in order to characterize the application complexity, computation and communication complexities of various class of algorithms should be modeled. Which is more fundamental, computation complexity or communication complexity? The communication complexity has greater impact on the execution complexity unlike the computation complexity. For example the computational complexity of Strassen's matrix multiplication is $O(N^{2.81})$, but has N the order of matrix increases its execution takes much longer time for completion than the traditional matrix multiplication. A closer analysis of the Strassen matrix multiplication algorithm exhibits, higher communication complexity unlike the traditional algorithm.

1. Application Complexity Design Space: What is lacking ?

Communication complexity of an algorithm gets mapped onto execution paths of varied lengths. To date there is no model to quantify the communication complexity of an algorithm and also there is no well defined metric for the same. As pointed out earlier, an application is a complex combination of a numeric, semi-numeric or non-numeric algorithms and general purpose logic and computing operations. Hence modeling the communication complexity of an application necessitates designing a quantitative model for communication complexity present in various classes of algorithms. To the best of knowledge of the authors, a quantitative model for communication complexity of an algorithm has not been dealt with to date. Of course there are computation complexity models for various classes of algorithms based on the concept presented in [3**]**.

2. The ALLDE based customizable simulator Design Space :

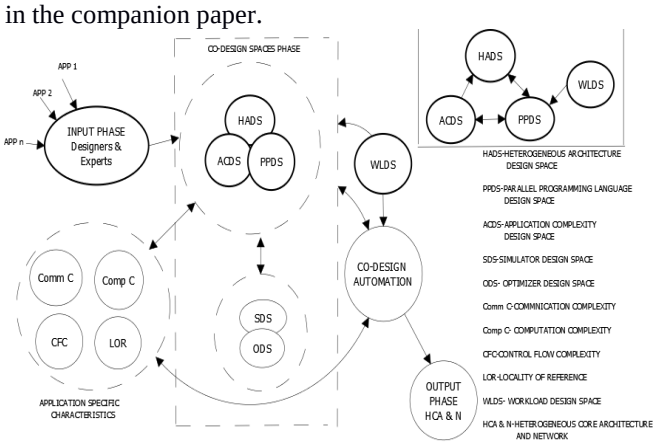
The typical simulators of the current day are totally incompatible with the concept of ALLDE based co-applications execution which is fundamental to achieve large scale utilization of enormous resources housed in thousands of cores. The proposed ALLDE based time model simulator is used to simulate the co-designed many core architecture for executing disparate workloads (co-applications) to determine the optimal intra-core components (like algorithm level functional units, the OCN components and the cache size),optimal number of core types and their respective count to maximize the performance given the power cap.

3. The ALLDE based customizable Optimizer Design Space:

Power-performance optimization, given the power cap, maximizing the performance, does not necessarily depend on the optimization techniques alone, it is equally important that how efficiently with reference to the target architecture, that is to be optimized, these techniques are applied to achieve the best. In the co-design of many core architecture for co-applications the optimization is applied at three levels during the formation of many core architecture as pointed out earlier unlike in the traditional co-design of many core architectures. This customizable optimizer is elaborated in the companion paper [107].

4. Many core co-design automation for co-applications :

Co-design automation process is an indispensable process to dominate over the co-design complexity to cut down the turnaround time. The co-design automation comprehends the dependencies across the many-core co-design spaces and devise the logics behind these interdependencies using a set of algorithms. This is It is essential that such co-design automation has a built in user customizable workload generator to benchmark the emerging many-core architecture. This customizability benefits the generation of complex workloads with the desired computation complexity, communication complexity, control flow complexity and locality of reference, specified under a distribution based on innovative quantitative models. The tremendous advantages of this customizable workload over the prevalent workload models and benchmark suits like SPEC, LINPACK, SPLASH2 and PARSEC are exemplified in the companion paper.



The binding across the application characteristics , namely the computational structures and

communication pattern(extracted from the application complexity design space) and the target many-core architecture(the corresponding functional units, algorithm level functional units and the scalars that get fixed in the many-core architecture design space). This in turn triggers the parallel programming design space to frame the respective parallel constructs. The afore mentioned design spaces together is called many-core co-design spaces. Refer Fig.3.

The SCOC IP cores are precast software building blocks using which innovative algorithm level functional units , scalars and network components can be created. These precast software building blocks can greatly bring down the turnaround time of complex architectures composed of thousands of cores of three to four types. These customizable constructs are very powerful and greatly reduce the complexity of developing an application code.

5. Many Core Architecture Design Space:

It is not just the matter of (device count per core) simple core or the complex core, the fact is how effectively the complexity is made use of (functionally) to maximize performance against power in a cost effective manner. With the number of applications increasing, the simple core seems to show better performance over the medium and complex cores and also overcomes the drawbacks due to Pollack rule. However these comparisons are not done using ALLDE model and the strongly bound co-design principles for many core architecture for co-applications proposed in this paper achieve tremendous power-performance efficiency and is shown in [107].Under the ALLDE model and the proposed strongly bound co-design many core architecture for co-applications, even the complex cores get very effectively exploited. There are four levels of inherent formation of heterogeneously networked many core architectures in the proposed concept of strongly bound co-design of many core architecture for co-applications unlike the prevalent co-design methodology.

a) Level 1: Based on the computational structures of the co-applications respective intra core components gets determined.
b) Level 2: Based on the computational and communication complexities of the co-applications the core types get formed .
c) Level: 3 Using level 2,the inter-core communication complexity, the respective count of the individual core types gets formulated.
d) Level: 4 Using level 3 the heterogeneous network for inter-core communication gets established.

The heterogeneity of many core architecture should get inherently formed leading to different types of core formation , by reflecting the heterogeneity of computational structures and communication complexity present in the co-applications, unlike the classic way of designing the many core architecture, just based only on architectural characteristics such as grouping of cores under different frequencies, under different types of scalar units, their respective count and a mix of out-of –order core and in –order core. Inherent formation of intra-core components, core types and their respective count towards the co-design of many core architectures for co-applications to bring about the strongest binding to maximize the performance, given the power cap is the fundamental departure from the classic co-design methodology which tumbles with the simple cores for a single complex application. Co-applications are constituted by different class of algorithms like numeric, semi-numeric and non-numeric and scalars. By analyzing the computational structures of the co- applications, intra core components the type of algorithm level functional units, the associated algorithm level instruction set architecture and the scalars are determined. An inherent formation of intra-core components, different types of cores along with their respective count inconformity with the characteristics of co-applications will lead to greater binding across the many core architecture and the co-applications.

V. CUSTOMIZABLE AND SCALABLE INTRA CORE ARCHITECTURE

The primary components of the intra-core architecture are algorithm level functional units, scalars, on core network, scheduler, dependency analyzer, loop-un-roller and cache. The customizability and scalability are such to balance the characteristics of the co-applications with each of the intra core architecture established on the MIMD model driven by the ALLDE programming model from the host. All these facets care discussed and conceptualized in this section.

1. Algorithm level Functional Units/Algorithm level Instruction Set:

Many target applications involve either numeric, semi-numeric, non-numeric or general purpose operations. These algorithm level functional units include units like 2*2 matrix multiplier,4*4 LU decomposer,8*8 transitive closure unit,32-point FFT, 8*8 minimum spanning tree unit and so on. power-performance efficiency of these units over the scalar based architecture has been established in and more so in depth the companion paper III, using the proposed XYZ simulator.

2. On-Core customizable Scheduler:

The job of the scheduler is to perform ALLDE efficiently. Under the concept of ALLDE, segments of different applications get executed within a same core. Also the instructions of the same type from different applications get simultaneously pipelined

within the same functional units. These instructions need to be identified and scheduled properly to appropriate functional units. For this purpose a hardware scheduler is designed to match with the parallel operation of hundreds of on chip cores. If an on chip scheduler is used for scheduling the instructions of the same type from the segments of different applications, to the pipelined stages of the same functional unit within a same core, it will be inefficient and also too complex. Hence an on core scheduler becomes imperative. Placing an on chip scheduler as in [nvidiacuda core, some 3 more references intelcmps,xstack] will lead to excessive on chip communication which is detrimental [Brokar, exascale references] even if it is for SIMD class of machines as in the GPUs[nvidia ref].

The parameters of the scheduler, the heuristics, the different tables and the buffer sizes(which depends on the peak count of parallel instructions to get executed within a core-ILP –design time parameter) are customizable. The scheduler architecture and its power-performance efficiency over the software scheduler is established in the companion paper III[107].

3. On-Core customizable Dependency analyzer:

Sequencing of the instructions from the multiple applications as per their respective inter dependency and assigning the time-stamps for their execution are required ahead of the scheduling process which performs the assignment of individual instructions to the corresponding algorithm level functional units/scalars of the core based on current execution status of multiple applications. Hence the time – stamp generator and the dependency analyzer are fundamental units for driving the scheduler which need to be housed on core for reasons given in the above sub-section. Architectures for these two units are actualized in the companion paper III.

4. On-Core customizable Loop-unroller:

Parallelizing the application program is an important aspect taking cognizant of the fact that many-core technology is heading towards the realization of hundreds of on chip cores for high performance computing system built around millions of cores. Several software components are being parallelized and new parallel languages [2 or 3 references], programming models and software kernals[ref] are getting developed in the background of great thrust towards co-design methodology for high performance computing. The most important software component, the program LOOPS also are not exempted. Of course parallelizing the loop has been a (Dongarra, J.J. and Hinds, A. R., “Unrolling Loops in Fortran”, *Software-Practice and Experience*, 9(3), Mar. 1979, pp. 219-226, Efficient Pipelining of Nested Loops:Unroll-and-Squash) long standing issue.

Several parallel algorithms for loop-unroller have been published [reference 3 or 4to be given].High performance architectures are designed to exploit instruction level parallelism and in such an environment loop-unrolling can bring to fore the independent instructions. An analysis shows that most of the program execution spend 10% to 30%of time in [An Aggressive Approach to Loop Unrolling] loops. It is said that 90% of the program time is spent in executing 10% of the program segments and these segments mostly have loops[Ref:An Aggressive Approach to Loop Unrolling].Loop-unrolling is likely to consume more power but it can definitely improve the performance**[**Ref**:**An Improving Method for Loop Unrolling].But there are certain types of loops which cannot be unrolled effectively, in the sense that it may not speed up the program execution[Ref:Loop Optimizations –QualifyingCandidates for Loop Unrolling]much.More on loop-unrolling algorithms are presented in the companion paperI[paper 1].Thus loop-unrolling has been gaining importance over a period of time span leading to their hardware implementation[Efficient Pipelining of Nested Loops:Unroll-and-Squash].Customizable hardware implementation of the loop-unroller is deliberated in the companion paper I [paper 1] and it is shown to be power-performance efficient than the corresponding software implementation. The parameters of the LU architecture, like the number of nested loops, the increment/decrement and terminating indices values and the terminating conditions can be fixed during the design time.

5. Customizable Hierarchical On Core Network:

Communication across the Algorithm Level Functional Units is initiated most of the time through the On Core Network (OCN) side stepping the cache levels. Power-performance efficiency of this novel approach is proved to be better by simulation results presented in this paper. The on core network is hierarchically based architecture having sub-local, local and global routers conceptualized in the companion paper II.The on core network parameters like number of router levels, and their buffers can be custom fixed during the design time. The importance of OCN is that all the algorithm level functional units/scalars can communicate through the OCN even bypassing the cache at that level. Simulation results show much increased cache hit over the current direct link between the cache and the functional units.

6. Customizable PCOS and SCOS:

Loop-unroller and the dependency analyzer are the primary components of a parallelizing

compiler[references].This leads to hardware based compiler and also conceptualized in the companion paper II with much more details simulation results. To tackle the hardware complexity, hierarchical compilation has been thought of, primary compiler on silicon(PCOS) and secondary compiler on silicon(SCOS), leaving the job of syntax and semantic analysis to the host system[paper 1].The architecture of PCOS and SCOS and their dynamics are conceptualized in.

VI. Parallel Programming Paradigm Design Space:

Co-Design has been an important factor in designing for high performance computing. This, as discussed in the previous sections, has been the approach taken by many of the researches undertaken and notably by [LLNL & INTEL Xstack]. While most of the focus has been on extracting the correct architecture for the desired application, the influence of the programming models has not been studied thoroughly. We present the Parallel Programming Design Space in the diagram shown below:

The Programming aspects have a strong influence in the design due to the high level of dependency on the architectural and application context. As can be inferred from the above diagram, the programming language is basically composed of the Computational, Memory and Control constructs. The Computational and communication constructs have a strong binding with the architecture in determining the word lengths that the programmer can operate with. The control constructs are generic and carry no dependency on the architecture or application. However, the level of control complexity and the need for specialised control constructs are based on the application under consideration. The memory constructs determine the storage aspects of data and depends on the type of storage model of the language, which is related to the nature of the application we design to solve. This, binding with the architecture and application, demands for the programming model to be included in the design space.

Existing programming models limit scalability, an important requirement for high performance computing, and hence needs to be updated [3]. Programming models present an abstraction of the underlying hardware and since there has been a huge growth in the hardware, there is a strong need for the programming models to be updated to adapt to the architectural advancements[UC Davies Slide]. Several programming languages like OpenMP, Chapel focus on parallelising the execution on just the memory constructs by using models like PGAS and MPI. However, lack of adaptive computational constructs , thereby limits the scalability of these languages [3 Survey Paper] making it hard to achieve higher performances.

An important factor for enabling high performance computing is customisability. Languages need to be customisable to suit to the needs of the application and architecture. Having a design space for the programming languages solves the problem of customisability. This however is found lacking in languages like Chapel, OpenMP, OpenCL etc. Another such factor is resource utilization. The above languages employ an implicit parallel model wherein the users are given the control of threads and parallelism. This however fails to completely utilise the resources as the user is unaware of how they get used. Thus, an explicit parallel model is suggested that lets the Hardware Compiler on Silicon (presented in the previous sections) allocate threads leading to an Architecture-aware threading model.

1. Customisable Parallel Language Paradigm:

Customisability and scalability are two most important factors needed in a parallel language as explained above. In this section we provide the features of a customisable and parallel programming language. As introduced earlier customisability of the language makes it suitable for high performance computing.

The Design space that is responsible for achieving customisability is detailed below

(a) Computational Constructs:

The computational constructs of the design space was constructed after a detailed analysis of the different classes of high performance computing applications. These applications may classified based on its computational structure into Numeric, Semi-Numeric and Non-Numeric algorithms.

The numeric class of algorithms refer to the applications that are purely mathematical in nature like LUD, SVD, Matmul, DFT, FFT. It may also be noted that many of these algorithms may further be defined as sub classes like Matrix Sub Classes, Transforms Sub Classes, and Series Sub Class. Such a classification of algorithms helps us easily determine the semantic aspects of the language.

The semi numeric classes consists of the algorithms that involve a few mathematical operations along with other types. The sorting sub class and the graph algorithms sub class comprise the semi numeric class of algorithms. The third type of class consists of algorithms that don't consists of mathematical operations. Matrix transpose and other permutation or combinatorial algorithms make up the non-numeric class of algorithms.

Thus the evolution of the programming language from such a wide range of options enables us to modify the design space with the application in focus.

(b) Communication And Memory Constructs:

The programming models are also defined based on the memory model being used. We propose a

slight modification to the existing models. The design space consists of the different memory models like shared memory, distributed memory, threaded model, Partitioned Global Address Model.

While any of the above models may be utilised we propose a slight modification to the PGAS model. The Dynamic Partitioned Global Address Space that we propose varies the memory space allocated to different applications dynamically by calculating their completion status and priorities. Thus several applications are allocated memory dynamically and exist simultaneously and making SMAPP possible. The performance of such a model is elaborated in the Results section of the paper.

VII. References

[1] Justin Solomon, "Numeric Algorithms", 2014, MIT Cambridge, https://people.csail.mit.edu/jsolomon/share/book/numerical_book.pdf

[2] Donald E Knuth, "The art of computer programming", 1997, Addison Wesley

[3] Donald E Knuth,"BIG OMICRON AND BIG OMEGA AND BIF THETA," in SIGACT news 1976

[4] Kushilevitz, Eyal. "Communication complexity." *Advances in Computers* 44 (1997): 331-360.

[5] Yao, Andrew Chi-Chih. "Some complexity questions related to distributive computing (preliminary report)." *Proceedings of the eleventh annual ACM symposium on Theory of computing*. ACM, 1979.

[6] Austin, Todd M., and Gurindar S. Sohi. "Dynamic dependency analysis of ordinary programs." *ACM SIGARCH Computer Architecture News*. Vol. 20. No. 2. ACM, 1992.

[7] Harman, Mark, et al. "Vada: A transformation-based system for variable dependence analysis." *Source Code Analysis and Manipulation, 2002. Proceedings. Second IEEE International Workshop on*. IEEE, 2002.

[8]

[9] Sadi, Muhammad Sheikh, Linkan Halder, and Seemanta Saha. "Variable dependency analysis of a computer program." *Electrical Information and Communication Technology (EICT), 2013 International Conference on*. IEEE, 2014.

[10] Sherstov, Alexander A. "The unbounded-error communication complexity of symmetric functions." *Combinatorica* 31.5 (2011): 583-614.

[11]

[12] Thompson, Clark D. "Area-time complexity for VLSI." *Proceedings of the eleventh annual ACM symposium on Theory of computing*. ACM, 1979.

[13] Cilardo, Alessandro, and Edoardo Fusella. "Design automation for application-specific on-chip interconnects: A survey." *Integration, the VLSI Journal* 52 (2016): 102-121.

[14] Henning, John L. "SPEC CPU2000: Measuring CPU performance in the new millennium." *Computer* 33.7 (2000): 28-35.

[15] Dongarra, Jack J., Piotr Luszczek, and Antoine Petitet. "The LINPACK benchmark: past, present and future." *Concurrency and Computation: practice and experience* 15.9 (2003): 803-820.

[16] Donofrio, David, et al. "Energy-efficient computing for extreme-scale science." *Computer* 42.11 (2009).

[17] PARKERe, John SHALF, and Li TANG. "On the role of co-design in high performance computing." *Transition of HPC Towards Exascale Computing* 24 (2013): 141.

[18] Bergman, Keren, et al. "Exascale computing study: Technology challenges in achieving exascale systems." *Defense Advanced Research Projects Agency Information Processing Techniques Office (DARPA IPTO), Tech. Rep* 15 (2008).

[19] Borkar, Shekhar. "Major challenges to achieve exascale performance." *Salishan Conference on High-Speed Computing*. 2009.

[20] Lebedev, Ilia, et al. "Exploring many-core design templates for FPGAs and ASICs." *International Journal of Reconfigurable Computing* 2012 (2012): 8.

[21] "Mohiyuddin, Marghoob. *Tuning hardware and software for multiprocessors*. University of California, Berkeley, 2012.

[22] Krueger, Jens, et al. "Hardware/software co-design for energy-efficient seismic modeling." *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2011.

[23] Mohiyuddin, Marghoob, et al. "A design methodology for domain-optimized power-efficient supercomputing." *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*. ACM, 2009.

[24] *"Customized Architectures", Center for Domain Specific Computing,UCLA*

[25] *RAMP: Accelerating Design of Next-Generation Supercomputers using Hardware Emulation,John Wawrzynek, Principal Investigator University of California, Krste AsanoviC (Co-PI) University of California, Berkeley*

[26] Cong, Jingsheng Jason. "Accelerator-rich architectures: from single-chip to datacenters." *Proceedings of the 2014 international symposium on Low power electronics and design*. ACM, 2014.

[27] Borkar, Shekhar. "Design perspectives on 22nm CMOS and beyond." *Proceedings of the 46th Annual Design Automation Conference*. ACM, 2009.

[28] Emerging Heterogeneous Technologies for High Performance Computing, Jack Dongarra, 22nd

International Heterogeneity in Computing Workshop, IPDPS, May 20, 2013

[29] Jason Cong, Zhenman Fang, Michael Gill, and Glenn Reinman. PARADE: A Cycle-Accurate Full System Simulation Platform for Accelerator-Rich Architectural Design and Exploration. The 2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD 2015), Austin, Texas, November 2015.

[30] **[**DAC12C**]** J. Cong, M.A. Ghodrat, M. Gill, B. Grigorian, and G. Reinman. Architecture support for accelerator-rich CMPs. Proceedings of the 49th Annual Design Automation Conference (DAC). June 2012.

[31] **[**ISLPED12C**]** J. Cong, M.A. Ghodrat, M. Gill, B. Grigorian and G. Reinman. CHARM: A Composable Heterogeneous Accelerator-Rich Microprocessor. Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED 2012), Redondo, California, pp. 379-384, July-August 2012

[32] **[**ISLPED13C**]** J. Cong, M.A. Ghodrat, M. Gill, B. Grigorian, H. Huang and G. Reinman. Composable Accelerator-rich Microprocessor Enhanced for Adaptivity and Longevity. Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED 2013), pp. 305-310, September 2013.

[33] **[**ISLPED12C**]** J. Cong, M.A. Ghodrat, M. Gill, C. Liu and G. Reinman. BiN: A Buffer-in-NUCA Scheme for Accelerator-Rich CMPs. International Symposium on Low Power Electronics and Design (ISLPED), Jul/Aug 2012.

[34] **[**DAC15C**]** Jason Cong, Michael Gill, Yuchen Hao, Glenn Reinman, and Bo Yuan. On-chip Interconnection Network for Accelerator-Rich Architectures. Design Automation Conference (DAC) 2015, San Francisco, CA, June 7-11, 2015.

[35] Single-ISA Heterogeneous Multi-Core Architectures:The Potential for Processor Power Reduction,Rakesh Kumar,Keith I. Farkas,Norman P. Jouppi_,Parthasarathy Ranganathan_,Dean M. Tullsen, Proceedings of the 36th International Symposium on Microarchitecture (MICRO-36 2003)

[36] Understanding Fundamental Design Choices in Single-ISA Heterogeneous Multicore Architectures, KENZO VAN CRAEYNEST and LIEVEN EECKHOUT, ACM Transactions on Architecture and Code Optimization, Vol. 9, No. 4, Article 32, Publication date: January 2013.

[37] Execution Migration in a Heterogeneous-ISA Chip Multiprocessor, Matthew DeVuyst Ashish Venkat Dean M. Tullsen

[38] NVIDIA's Next Generation CUDA TM Compute Architecture: Fermi

[39] Heterogeneous Chip Multiprocessors, Rakesh Kumar, Dean M.Tullsen, Norman P.Jouppi,Parthasarathy Ranganathan, IEEE Computer 2005, Published by the IEEE Computer Society